

Sistemas Distribuidos II



TicketApp es un sistema de venta de tickets para eventos. Durante este cuatrimestre construirás un sistema distribuido completo que permite: vender tickets online, generar códigos QR para validación y validar entrada a eventos, aplicando los conceptos teóricos vistos en clase.

Esta primera práctica establece el microservicio central: orders-service, que gestiona las órdenes de compra de tickets.

Objetivos de Aprendizaje

- Implementar API REST con Spring Boot
- Aplicar arquitectura de capas (Controller-Service-Repository)
- Implementar operaciones CRUD completas
- Manejar validaciones y errores de forma consistente
- Aplicar buenas prácticas de diseño de APIs

Requisitos

- Java 17+ instalado
- Maven o Gradle



- IDE (IntelliJ IDEA recomendado)
- Postman o similar para testing

Parte 1: Setup del Proyecto

1.1 Crear Proyecto

- Usar Spring Initializr (start.spring.io) con:
- Project: Gradle
- Language: Java
- Spring Boot: (última estable)
- Group: com.TicketApp
- Artifact: orders-service

1.2 Dependencias

- Incluir las siguientes dependencias:
- spring-boot-starter-web
- Se pueden agregar más dependencias si se requieren.

1.3 Estructura de Paquetes

```
com.TicketApp.orders
├── controller/
├── service/
├── repository/
├── model/
├── dto/
└── exception/
```

Parte 2: Modelo de Datos

2.1 Entidad Order

Crear la entidad Order con los siguientes campos:

Campo	Tipo	Descripción
id	Long	Identificador único (auto-generado)
eventId	Long	ID del evento asociado
eventName	String	Nombre del evento (desnormalizado)
buyerEmail	String	Email del comprador
buyerName	String	Nombre del comprador
quantity	Integer	Cantidad de tickets
unitPrice	BigDecimal	Precio por ticket
totalAmount	BigDecimal	Monto total
status	OrderStatus	Estado de la orden
qrCode	String	Código QR (nullable, se genera async)
createdAt	LocalDateTime	Fecha de creación
updatedAt	LocalDateTime	Fecha de actualización

2.2 Enum OrderStatus

Estados posibles de una orden:

- PENDING: orden creada, esperando confirmación
- CONFIRMED: pago confirmado, QR generado
- CANCELLED: orden cancelada



- USED: ticket ya fue utilizado (validado en entrada)

2.3 DTOs

Crear DTOs para separar la representación externa del modelo interno:

- OrderCreateRequest: datos para crear orden
- OrderResponse: respuesta con datos de orden
- OrderUpdateRequest: datos para actualizar orden

Parte 3: Capa de Repositorio

3.1 OrderRepository

Para crear los repositorios: se puede usar “JpaRepository” con queries custom:

```
public interface OrderRepository extends JpaRepository<Order, Long> {  
    List<Order> findByBuyerEmail(String email);  
    List<Order> findByEventId(Long eventId);  
    List<Order> findByStatus(OrderStatus status);  
    Optional<Order> findByQrCode(String qrCode);  
}
```

Parte 4: Capa de Servicio

4.1 OrderService

Implementar la lógica de negocio:

```
createOrder(): validar datos, calcular total, guardar con status PENDING  
getOrderById(): obtener orden o lanzar excepción  
getOrdersByBuyer(): listar órdenes de un comprador  
updateOrderStatus(): cambiar estado con validaciones de transición  
cancelOrder(): solo si está PENDING
```



```
validateTicket(): buscar por QR y marcar como USED
```

4.2 Validaciones de Negocio

Implementar las siguientes reglas:

- quantity debe ser entre 1 y 5
- buyerEmail debe ser email válido
- No se puede cancelar orden CONFIRMED o USED
- No se puede usar ticket ya USED o CANCELLED

Parte 5: Capa de Controller

5.1 OrderController

Método	Path	Descripción
POST	/api/orders	Crear nueva orden
GET	/api/orders/{id}	Obtener orden por ID
GET	/api/orders	Listar órdenes (con filtros)
PUT	/api/orders/{id}	Actualizar orden
DELETE	/api/orders/{id}	Cancelar orden
POST	/api/orders/validate/{qrCode}	Validar ticket por QR

5.2 Filtros de Búsqueda

El endpoint GET /api/orders debe soportar:

- ?buyerEmail=email@example.com
- ?eventId=123
- ?status=CONFIRMED



Parte 6: Manejo de Errores

6.1 Excepciones Custom

Crear excepciones específicas, considerar los response code correspondientes:

- OrderNotFoundException: orden no existe
- InvalidOrderStateException: transición de estado inválida
- TicketAlreadyUsedException: ticket ya fue validado

6.2 Estructura de Error

Respuestas de error deben seguir estructura consistente:

```
{
  "timestamp": "2024-01-15T10:30:00",
  "status": 404,
  "error": "Not Found",
  "message": "Order with id 123 not found",
  "path": "/api/orders/123"
}
```

Entregables

Link al classroom de la tarea: <https://classroom.github.com/a/r7K9ZE7c>

- Código fuente completo del orders-service
- Colección Postman con ejemplos de cada endpoint (crear alguna carpeta de documentación)
- README con instrucciones para su ejecución local. Version Java y springboot usada.

Fecha de entrega: 2026-03-26 6:00 PM

